Department of Computer and Software Engineering
SE: Machine Learning

Course Instructor:	**Date:**
Day:	**Semester:**

Lab 11: Refactoring, MLflow and Deployment

Lab Title	CLO	BT	PLO	Weightage
Refactoring & Deployment MLflow, APIs, Serialization				

Name	Roll Number	Report /10	Viva /5	Total /15

Checked on: _____

Signature: _____

Objectives

- Refactor unstructured ML code into modular functions
- Track experiments using MLflow
- Serialize trained models
- Build REST APIs using FastAPI

Problem Statement

In real-world machine learning systems, models are not built as isolated scripts but as part of structured, maintainable, and deployable pipelines.

You are given a poorly written ("spaghetti") machine learning script that mixes data loading, preprocessing, model training, and evaluation in a single block of code. Such code is difficult to maintain, extend, debug, and deploy.

Your task is to transform this code into a clean and modular pipeline by separating concerns into well-defined functions.

After refactoring, you will extend the pipeline to include:

- **Experiment Tracking:** Use MLflow to log model parameters, performance metrics, and artifacts.
- **Model Serialization:** Save the trained model to disk and reload it for inference.
- **Model Deployment:** Build a REST API using FastAPI to serve predictions from the trained model.

You will use the dataset from the previous lab (Lab 10) to ensure continuity.

By the end of this lab, you will have a complete mini machine learning system that includes:

- Clean, modular code
- Experiment tracking
- Persistent models
- A working prediction API

Part A: Refactoring

Task 1: Identify Issues
Open `train_bad.py` and list at least four problems related to structure and maintainability (e.g. hard-coded paths, no functions, magic numbers, mixed concerns).

Task 2: Modularization
Refactor the script into the following functions inside `lab11_starter.py`:

- `load_data()` — read `lab10_data.csv` and return a `DataFrame`.
- `preprocess_data(df)` — drop the categorical `city` column, then split into feature matrix `X` (all columns except `target`) and label vector `y`.
- `train_model(X, y)` — fit and return a `DecisionTreeClassifier`.
- `evaluate_model(model, X, y)` — compute and return the accuracy score (expected ≥ 0.8).
- `run_pipeline()` — orchestrate the above steps and save the model to disk.

Part B: MLflow

Task 3: Logging
Inside `run_pipeline()`, wrap the training and evaluation steps in an MLflow run and log the following:

- **Parameters:** model hyper-parameters (e.g. `max_depth`, `criterion`).
- **Metrics:** training accuracy (key: `accuracy`; must reach the `expected_accuracy` threshold of 0.8).
- **Model:** the fitted `DecisionTreeClassifier` via `mlflow.sklearn.log_model`.

Part C: Serialization

Task 4: Save and Load Model
Use `joblib` to persist the trained model:

- In `run_pipeline()`: call `joblib.dump(model, "model.joblib")` after training.
- In the FastAPI section: call `joblib.load("model.joblib")` to restore the model before the app starts serving requests.

Part D: FastAPI

Task 5: API Endpoint
Complete the `/predict` POST endpoint in `lab11_starter.py`:

- **Request body:** {`"features": [age, income, hour, leak_feature]`}
- **Response:** {`"prediction": 0`} or {`"prediction": 1`}
- Raise `HTTPException(status_code=422)` if the `features` key is missing.
- Raise `HTTPException(status_code=503)` if the model has not been loaded.

Verify the API is reachable via the `GET /` endpoint which returns {`"message": "ML Model API is running"`}.

Discussion Questions

- Why is modular design important?
- What are benefits of MLflow?
- Why is model serialization required?